



**UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA
ESPECIALIZAÇÃO EM RESIDÊNCIA EM TECNOLOGIA DA INFORMAÇÃO**

Samuel Victor Oliveira Lima

**EJUG NOTIFY: PLATAFORMA INSTITUCIONAL PARA NOTIFICAÇÕES
OFICIAIS VIA WHATSAPP BUSINESS PLATFORM**

Trabalho de Conclusão de Curso – Gate 3: Desenvolvimento e Escrita

Goiânia
2026

UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA
ESPECIALIZAÇÃO EM RESIDÊNCIA EM TECNOLOGIA DA INFORMAÇÃO

Samuel Victor Oliveira Lima

**EJUG NOTIFY: PLATAFORMA INSTITUCIONAL PARA NOTIFICAÇÕES
OFICIAIS VIA WHATSAPP BUSINESS PLATFORM**

Trabalho de Conclusão de Curso apresentado ao Programa de Pós-Graduação Lato Sensu Especialização em Residência em Tecnologia da Informação do Instituto de Informática da Universidade Federal de Goiás, como requisito parcial para obtenção do título de Especialista em Residência em Tecnologia da Informação.

Orientador(a): Prof. Leonardo Antônio Alves

RESUMO EXECUTIVO

Este documento apresenta o rascunho do Gate 3 do projeto EJUG Notify, uma plataforma institucional para cadastro de contatos, turmas, templates e campanhas de notificação via WhatsApp Business Platform, voltada ao contexto da Escola Judicial de Goiás. O problema observado é a dificuldade de comunicação com turmas numerosas, em cenários nos quais soluções baseadas em automação de WhatsApp Web ou uso de chips convencionais geram risco de bloqueio por comportamento semelhante a spam. A proposta deste trabalho é substituir a lógica de disparo informal por uma solução oficial, auditável e escalável, baseada em opt-in, templates aprováveis, segmentação por turmas, filas de processamento, webhooks, registros de auditoria e controles de segurança.

No estágio atual, o projeto contempla uma base executável com API Node.js/Express, MongoDB, Redis/BullMQ, frontend Vue 3/Vite, Docker Compose e documentação técnica. A aplicação foi desenhada para não quebrar quando as variáveis da Meta estiverem vazias: nesse cenário, o frontend exibe aviso visual, os fluxos de cadastro continuam funcionando e apenas o envio real pela WhatsApp Cloud API permanece bloqueado com mensagem amigável. O repositório do projeto está disponível em: <https://github.com/samuelvictorol/tcc-prti-samuelvictor>.

LISTA DE FIGURAS

Figura 1 – Tipos de fontes utilizadas como referência.

LISTA DE QUADROS E TABELAS

Os quadros e tabelas deste rascunho apresentam arquitetura, módulos, entidades, segurança, execução local e roadmap técnico.

LISTA DE SIGLAS E ABREVIATURAS

API	Application Programming Interface
DTO	Data Transfer Object
EJUG	Escola Judicial de Goiás
LGPD	Lei Geral de Proteção de Dados Pessoais
MVP	Minimum Viable Product
PRTI	Programa de Residência em Tecnologia da Informação
TJGO	Tribunal de Justiça do Estado de Goiás
WABA	WhatsApp Business Account

Sumário

RESUMO EXECUTIVO	3
LISTA DE FIGURAS	4
LISTA DE QUADROS E TABELAS	5
LISTA DE SIGLAS E ABREVIATURAS	6
1 INTRODUÇÃO	8
1.1 PROBLEMA	8
1.2 OBJETIVO GERAL	8
1.3 OBJETIVOS ESPECÍFICOS	8
1.4 JUSTIFICATIVA	9
2 FUNDAMENTAÇÃO TEÓRICA	10
2.1 FONTES DE REFERÊNCIA	10
2.2 WHATSAPP BUSINESS PLATFORM	10
2.3 TEMPLATES, CONSENTIMENTO E LIMITES	10
2.4 FILAS E PROCESSAMENTO ASSÍNCRONO	11
2.5 LGPD E GOVERNANÇA DE DADOS	11
3 METODOLOGIA	12
3.1 LEVANTAMENTO E PRIORIZAÇÃO	12
3.2 CRITÉRIOS DE DESENVOLVIMENTO	12
4 DESENVOLVIMENTO	13
4.1 ARQUITETURA DE NEGÓCIO	13
4.2 PROTÓTIPO OU MVP	13
4.3 ARQUITETURA TÉCNICA	14
4.4 MODELO DE DADOS	14
4.5 FLUXO DE ENVIO	15

4.6	SEGURANÇA E CONFORMIDADE	16
4.7	ESTRUTURA DO REPOSITÓRIO	16
4.8	EXECUÇÃO LOCAL E VARIÁVEIS DE AMBIENTE	17
4.9	ROADMAP DE EVOLUÇÃO	18
5	CONSIDERAÇÕES FINAIS	19
	REFERÊNCIAS	20
	APÊNDICES	21
	ANEXOS	22

1 INTRODUÇÃO

A comunicação institucional com grandes grupos de pessoas é uma necessidade recorrente em escolas judiciais, órgãos públicos e instituições de ensino. No contexto da Escola Judicial de Goiás, uma dificuldade relatada envolve o envio de notificações, lembretes e avisos para turmas com diferentes volumes de participantes, variando de pequenos grupos até bases com milhares de contatos. Soluções previamente testadas, baseadas em softwares de disparo e uso de chips convencionais, apresentaram risco de bloqueio por envio em massa, o que compromete a confiabilidade do processo.

O presente trabalho propõe o desenvolvimento da plataforma EJUG Notify, uma solução de software para gerenciar contatos, turmas, templates de mensagens e campanhas de comunicação institucional via WhatsApp Business Platform. A proposta evita o uso de automações baseadas em WhatsApp Web, QR Code ou simulação de comportamento humano. Em vez disso, adota uma arquitetura baseada na API oficial, com filas, consentimento, templates, logs, segurança e rastreabilidade.

1.1 PROBLEMA

Como permitir que a EJUG envie notificações e lembretes para turmas numerosas de forma escalável, institucional, auditável e menos suscetível a bloqueios por spam?

1.2 OBJETIVO GERAL

Desenvolver uma primeira versão funcional de uma plataforma institucional para cadastro de contatos, organização de turmas, criação de templates e preparação de campanhas de notificação via WhatsApp Business Platform, com arquitetura segura, documentada e executável em ambiente local.

1.3 OBJETIVOS ESPECÍFICOS

- modelar os dados principais da solução, incluindo contatos, turmas, templates, campanhas, mensagens e consentimentos;
- implementar backend com separação entre controllers, managers, DTOs, normalizers e services;
- implementar frontend Vue 3/Vite moderno e responsivo para visualização dos fluxos administrativos;

- criar Docker Compose para execução local do backend, frontend, banco e fila;
- permitir uso do sistema mesmo sem credenciais da Meta, bloqueando apenas o envio real;
- documentar arquitetura, segurança, fluxos, execução local e metodologia para o Gate 3;
- disponibilizar o repositório de código inicial do projeto para validação técnica.

1.4 JUSTIFICATIVA

O projeto é relevante porque aborda uma necessidade prática de comunicação institucional em escala, com foco em conformidade e governança. Ao substituir ferramentas frágeis de disparo por uma solução baseada em API oficial, o projeto reduz riscos operacionais, melhora a rastreabilidade e cria uma base técnica que pode ser evoluída para implantação real em ambiente institucional.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 FONTES DE REFERÊNCIA

A formalização teórica do projeto utiliza diferentes tipos de fontes, incluindo documentação técnica oficial, legislação aplicável, materiais acadêmicos e documentos institucionais. A Figura 1 apresenta uma referência visual sobre categorias de fontes utilizadas em trabalhos técnicos e científicos.

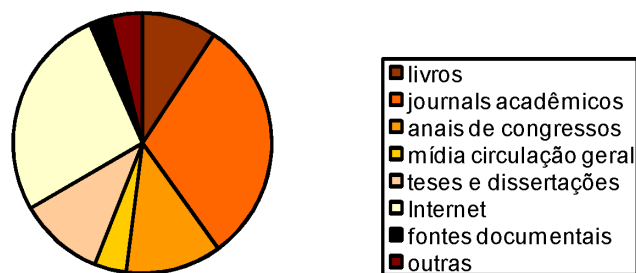


Figura 2.1 – Tipos de fontes utilizadas como referência

Fonte: Template oficial adaptado para o Gate 3.

2.2 WHATSAPP BUSINESS PLATFORM

A WhatsApp Business Platform é a solução oficial da Meta para integração de sistemas empresariais ao WhatsApp. Por meio da Cloud API, aplicações podem enviar mensagens de forma programática, configurar webhooks e integrar fluxos de atendimento ou notificação. A documentação oficial da Meta descreve o processo de criação do aplicativo, configuração do número, envio de mensagens e uso de webhooks.

No contexto deste projeto, a plataforma é utilizada como referência para uma solução institucional. O objetivo não é automatizar uma sessão de navegador ou parear um celular por QR Code, mas sim criar uma aplicação que se comunique com a infraestrutura oficial por meio de tokens, identificadores de conta e número, webhooks e templates.

2.3 TEMPLATES, CONSENTIMENTO E LIMITES

Mensagens institucionais enviadas fora de uma conversa iniciada pelo usuário dependem de templates previamente estruturados e adequados à política da plataforma. O projeto adota a categoria de mensagens utilitárias para lembretes, avisos de aula, confirmação de inscrição,

alteração de horário e disponibilização de certificado. Também incorpora a exigência de opt-in e opt-out, permitindo registrar quando o participante autorizou o recebimento e quando solicitou cancelamento.

2.4 FILAS E PROCESSAMENTO ASSÍNCRONO

Para lidar com volumes elevados, o envio não deve ocorrer de forma síncrona a partir de uma requisição HTTP. A arquitetura proposta utiliza uma fila baseada em Redis/BullMQ, permitindo enfileirar cada mensagem como um job individual, aplicar controle de vazão, tentativas de reproprocessamento e atualização de status. Essa abordagem melhora a resiliência e evita que a interface administrativa fique bloqueada durante campanhas com muitos destinatários.

2.5 LGPD E GOVERNANÇA DE DADOS

Como o sistema manipula nomes, telefones, e-mails e registros de consentimento, a solução deve considerar princípios da Lei Geral de Proteção de Dados Pessoais, incluindo finalidade, necessidade, transparência e segurança. No MVP, a aplicação já separa DTOs para evitar exposição desnecessária de dados e registra eventos de opt-in e opt-out como parte da trilha de auditoria.

3 METODOLOGIA

O desenvolvimento foi conduzido como uma prova de conceito incremental, voltada à transformação da ideia em uma estrutura técnica concreta. A metodologia combinou levantamento do problema, definição de requisitos, modelagem de dados, separação arquitetural, implementação de módulos iniciais e validação local por Docker Compose.

3.1 LEVANTAMENTO E PRIORIZAÇÃO

O problema principal foi priorizado a partir da demanda de comunicação em massa da EJUG e da limitação observada em soluções anteriores. A restrição mais importante identificada foi evitar abordagens não oficiais baseadas em chips e automação de WhatsApp Web, priorizando uma solução institucional, escalável e auditável.

3.2 CRITÉRIOS DE DESENVOLVIMENTO

Foram definidos os seguintes critérios técnicos:

- separação entre camada HTTP e regra de negócio;
- controllers responsáveis apenas pela entrada e saída das requisições;
- managers responsáveis pelas regras de negócio;
- DTOs para padronizar retorno e reduzir tráfego desnecessário;
- normalizers isolados para telefone, texto e paginação;
- services isolando integrações externas, como a WhatsApp Cloud API;
- uso de fila para processamento assíncrono;
- Docker Compose para reprodutibilidade local;
- bloqueio seguro do envio real caso as credenciais da Meta não estejam configuradas.

4 DESENVOLVIMENTO

4.1 ARQUITETURA DE NEGÓCIO

A solução proposta atende a três atores principais: administrador da EJUG, participante/aluno e infraestrutura de mensageria oficial. O administrador gerencia turmas, contatos, templates e campanhas. O participante recebe notificações institucionais mediante consentimento. A plataforma oficial do WhatsApp atua como provedor de envio e retorno de status.

QUADRO 4.1 – Visão de negócio da solução

Elemento	Descrição
Problema	Bloqueio de chips e instabilidade em disparos para grandes bases de contatos.
Solução	Plataforma institucional baseada em WhatsApp Business Platform, opt-in, templates e fila.
Usuário principal	Equipe administrativa da EJUG.
Beneficiário	Alunos e participantes de cursos, eventos e formações.
Valor gerado	Comunicação mais confiável, rastreável, segmentada e compatível com ambiente institucional.

4.2 PROTÓTIPO OU MVP

O MVP implementado para o Gate 3 contém backend e frontend executáveis localmente. A aplicação permite login administrativo, cadastro de contatos, registro de opt-in, cadastro de turmas, associação de contatos a turmas, cadastro de templates, criação de campanhas e processamento por fila. O envio real via WhatsApp Cloud API permanece condicionado à configuração das variáveis oficiais da Meta.

O projeto entrega API Node.js/Express com MongoDB, frontend Vue 3/Vite moderno e responsivo, Redis/BullMQ para fila de mensagens, Docker Compose com API, frontend, MongoDB e Redis, estrutura com controllers, managers, DTOs, normalizers, services e models, além de integração preparada para Meta WhatsApp Cloud API. O app foi preparado para não quebrar caso as variáveis da Meta estejam vazias.

4.3 ARQUITETURA TÉCNICA

A arquitetura foi organizada em duas aplicações principais: API e frontend. A API utiliza Express, MongoDB e Redis/BullMQ. O frontend utiliza Vue 3 com Vite para demonstrar os fluxos administrativos.

QUADRO 4.2 – Arquitetura geral da aplicação

Camada	Responsabilidade
Frontend Vue 3/Vite	Interface administrativa responsiva para login, dashboard, contatos, turmas, templates, campanhas e notificação rápida.
API Express	Exposição dos endpoints REST e autenticação administrativa.
Controllers	Recebem requisições, chamam managers e retornam respostas padronizadas.
Managers	Concentram regras de negócio, validações e orquestração.
DTOs	Padronizam os dados trafegados nos endpoints e evitam exposição excessiva.
Normalizers	Tratam telefone, texto e paginação sem poluir regras de negócio.
Services	Isolam integrações externas, especialmente a comunicação com a Meta.
MongoDB	Persistência de contatos, turmas, campanhas, templates, mensagens e consentimentos.
Redis/BullMQ	Fila assíncrona para envio de mensagens.
WhatsApp Cloud API	Provedor oficial de envio e retorno de eventos via webhook.

4.4 MODELO DE DADOS

Tabela 4.1 – Principais modelos de dados

Modelo	Finalidade
User/Admin	Usuário administrativo com nome, e-mail, senha criptografada e perfil de acesso.

Contact	Representa aluno/participante, com nome, telefone normalizado, e-mail, documento, origem e informações de opt-in/opt-out.
Group	Representa turma, curso ou grupo institucional de segmentação, com contatos vinculados.
MessageTemplate	Modelo de mensagem com nome interno, categoria, idioma, corpo, variáveis, status e nome de template aprovado na Meta.
Campaign	Campanha de comunicação associada a um grupo e template, com status, agendamento, responsável e contadores.
MessageLog	Unidade individual de envio para um contato, com status, telefone, mensagem, identificador do provedor e datas de envio, entrega ou leitura.
ConsentLog	Registro de consentimento, origem, opt-in, opt-out e metadados de auditoria.

4.5 FLUXO DE ENVIO

QUADRO 4.3 – Fluxo técnico de campanha

Etapa	Descrição
1	Administrador cadastra contatos e registra opt-in.
2	Administrador cria turma e associa contatos.
3	Administrador cadastra template de notificação.
4	Administrador cria campanha vinculando turma e template.
5	Sistema prepara envios somente para contatos ativos, com opt-in e sem opt-out.
6	Jobs são enfileirados no Redis/BullMQ com controle de vazão.
7	Worker processa cada job e chama a WhatsApp Cloud API, se configurada.
8	Webhook atualiza status de mensagens entregues, lidas ou falhas.
9	Dashboard exibe contadores de campanha e rastreabilidade.

4.6 SEGURANÇA E CONFORMIDADE

QUADRO 4.4 – Controles de segurança previstos no MVP

Controle	Aplicação no projeto
Autenticação	Login administrativo com JWT e senha criptografada.
DTOs	Retorno controlado de dados nos endpoints.
Opt-in	Registro de consentimento antes do envio.
Opt-out	Palavras como SAIR, PARAR, CANCELAR e STOP removem o contato de futuras campanhas.
Webhook verify token	Token de verificação para validar callback da Meta.
Variáveis de ambiente	Tokens, chaves e IDs sensíveis ficam fora do código-fonte.
Falha segura	Sem variáveis da Meta, o sistema não quebra e bloqueia apenas o envio real.
Auditoria	Registros de consentimento, status e eventos permitem rastreabilidade.

4.7 ESTRUTURA DO REPOSITÓRIO

O repositório foi organizado para separar backend, frontend, documentação e arquivos de execução local. A estrutura principal é apresentada a seguir.

```
/api
  /src
    /config
    /controllers
    /dtos
    /enums
    /errors
    /jobs
    /managers
    /middlewares
    /models
    /normalizers
    /queues
    /routes
    /services
```

```
  /utils
/frontend
  /src
    /assets
    /boot
    /components
    /composables
    /layouts
    /pages
    /router
    /services
    /utils
/docs
docker-compose.yml
.env
.env.example
README.md
```

4.8 EXECUÇÃO LOCAL E VARIÁVEIS DE AMBIENTE

A execução local foi preparada para facilitar a validação do Gate 3 em diferentes máquinas. O ambiente sobe API, frontend, MongoDB e Redis por meio do Docker Compose.

```
docker compose down -v
docker compose up
```

Após os containers iniciarem, o administrador inicial pode ser criado com o comando:

```
docker compose exec api npm run seed
```

O acesso local fica disponível em:

```
Frontend: http://localhost:5173
API Health: http://localhost:3000/api/health
Login: admin@ejug.local
Senha: admin123
```

No desenvolvimento local, as variáveis abaixo podem permanecer vazias sem quebrar o sistema:

```
WHATSAPP_PHONE_NUMBER_ID=
WHATSAPP_BUSINESS_ACCOUNT_ID=
```

```
WHATSAPP_ACCESS_TOKEN=
```

Com elas vazias, o frontend mostra aviso visual, cadastros continuam funcionando, templates continuam funcionando, turmas continuam funcionando e campanhas podem ser criadas em modo sem envio real. O envio efetivo pela Meta fica bloqueado com mensagem amigável.

Caso algum ambiente utilize Docker BuildKit e apresente travamento na etapa de provenance, pode-se desabilitar temporariamente o BuildKit para fins de teste local:

```
DOCKER_BUILDKIT=0 COMPOSE_DOCKER_CLI_BUILD=0 docker compose up
```

4.9 ROADMAP DE EVOLUÇÃO

QUADRO 4.5 – Roadmap técnico

Fase	Entregas previstas
Gate 3	Estrutura técnica, modelos de dados, API, frontend Vue 3, Docker Compose, documentação e repositório inicial.
Gate 4	Integração real com Meta, templates oficiais, testes de webhook e relatório de validação.
Piloto	Uso controlado com turma real, métricas de entrega, opt-out e feedback dos usuários.
Produção	Segurança avançada, perfis de acesso, logs completos, dashboard analítico e plano de sustentação.
Escala	Replicação para outras escolas judiciais ou tribunais, com licenciamento e governança multi-instituição.

5 CONSIDERAÇÕES FINAIS

O Gate 3 transforma a ideia inicial em uma estrutura concreta de software e documentação. O projeto já possui uma base técnica executável, com separação de responsabilidades, modelos de dados, interface Vue 3, ambiente local via Docker, fila Redis/BullMQ e organização de código preparada para evolução. A solução demonstra viabilidade para substituir práticas frágeis de disparo em massa por uma arquitetura mais segura, institucional e alinhada ao uso oficial do WhatsApp Business Platform.

Como limitação desta etapa, o envio real ainda depende da configuração das credenciais da Meta, aprovação de templates e eventual número institucional. A próxima etapa do projeto deverá validar a integração real com a Cloud API, configurar webhooks externos por túnel ou ambiente publicado, testar templates aprovados e evoluir o dashboard de métricas.

REFERÊNCIAS

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018.** Lei Geral de Proteção de Dados Pessoais. Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 22 maio 2026.

META. **WhatsApp Cloud API: Get Started.** Meta for Developers. Disponível em: <https://developers.facebook.com/documentation/business-messaging/whatsapp/get-started>. Acesso em: 22 maio 2026.

META. **WhatsApp Business Platform: Messaging Limits.** Meta for Developers. Disponível em: <https://developers.facebook.com/documentation/business-messaging/whatsapp/messaging-limits>. Acesso em: 22 maio 2026.

META. **WhatsApp Business Platform Webhooks.** Meta for Developers. Disponível em: <https://developers.facebook.com/documentation/business-messaging/whatsapp/webhooks/overview>. Acesso em: 22 maio 2026.

OLIVEIRA LIMA, Samuel Victor. **tcc-prti-samuelvictor: repositório do projeto EJUG Notify.** GitHub, 2026. Disponível em: <https://github.com/samuelvictorol/tcc-prti-samuelvictor>. Acesso em: 22 maio 2026.

WHATSAPP. **WhatsApp Business Messaging Policy.** Disponível em: <https://whatsappbusiness.com/policy/>. Acesso em: 22 maio 2026.

APÊNDICES

Apêndice A – Estrutura inicial do repositório contendo backend, frontend, Docker Compose e documentação técnica.

Apêndice B – Instruções de execução local e criação do administrador inicial.

Apêndice C – Repositório do projeto: <https://github.com/samuelvictorol/tcc-prti-samuelvictorol>

ANEXOS

Anexo A – Logotipo institucional da UFG utilizado na capa.

Anexo B – Figura de referência sobre tipos de fontes utilizadas no trabalho.